A Report on

License Plate Recognition System using CNN

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Engineering

by

Miss. Tejal V. Gawade (17)
Miss. Rutuja S. Parab (39)
Miss. Tejasvi R. Parab (41)

Guide:
Prof. D. P. Mhapasekar



DEPARTMENT OF COMPUTER ENGINEERING

Sindhudurg Shikshan Prasarak Mandals's College of Engineering. (Kankavli)

(2024-25)

CERTIFICATE

This is to certify that the project entitled **License Plate Recognition System using CNN** is a bonafide work of **Miss. Tejal V. Gawade (17), Miss. Rutuja S. Parab (39), Miss. Tejasvi R. Parab (41)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Engineering**.

Prof. D. P. Mhapasekar
Supervisor/Guide

Prof. S. B. Kadam
Project Coordinator

Prof. D. P. Mhapasekar
Head of Department

Dr. D. S. Badkar
Principal

Project Approval Sheet

This project entitled ***License Plate Recognition System using CNN*** by ***Miss. Tejal V. Gawade, Miss. Rutuja S. Parab, Miss. Tejasvi R. Parab*** is approved for the degree of **Bachelor of Computer Engineering**.

Examiners

1.———————————-

2.———————————-

Date:

Place: Kankavli.

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name and signature of students:

1. Miss. Tejal V. Gawade

2. Miss. Rutuja S. Parab

3. Miss. Tejasvi R. Parab

Date :

Abstract

License Plate Recognition has become an important aspect in our daily life because of unlimited increase of vehicles and transportation system. It is more and more difficult to fully manage and monitor by humans. Advancements in acquisition devices have increased the availability of high definition (HD) images, which can capture images of multiple vehicles. License Plate Recognition Systems have been developed to identify vehicles in real-time situations. Since license plate (LP) occupies a relatively small portion of an image, therefore detection of LP in an image is considered a challenging task, as it is difficult to locate a small object in an entire image. Automatic license plate recognition which has many applications in intelligent transportation systems has three main steps, License Plate Detection (LPD), segmentation, and character recognition. Due to the various types of license plates formats, varying scales and sizes, different angles, illumination changes, etc. this is quite a challenging problems. In this, an algorithm for LPD is proposed. In the proposed technique, to remove noises from input images mean filtering is used. After that, the differences between the filtered image and input image are computed and it is converted to a black and white image. Finally, after applying edge detection and closing and opening morphological operations, license plate candidates are detected. Thereafter, character recognition methodology has been applied to identify characters from the number plate. For automatic detection of a license plate from an image, which is followed by plate correction, or in other words, plate rectification. Convolutional Neural Networks (CNNs) based approach is used to locate corner points of license plate image, after that plate rectification done using perspective transformation. CNN is used for recognition alphanumeric characters from License Plates.

Contents

Abstract v

1 Introduction 1
- 1.1 Existing Systems . . . 1
- 1.2 Limitations of Existing system . . . 2
- 1.3 Proposed system . . . 3
- 1.4 Project Scope . . . 3

2 Literature Review 5
- 2.1 Summary of research papers . . . 5
- 2.2 Research Gap . . . 5
- 2.3 Problem Statement . . . 6
  - 2.3.1 Project Goals . . . 6

3 Methodology 7
- 3.1 System Architecture . . . 7
- 3.2 Process Description . . . 8

4 Experimental Setup 12
- 4.1 Details About Input to Systems . . . 12
- 4.2 Performance Evaluation Parameters . . . 12
- 4.3 Software and Hardware Setup . . . 13

5 Implementation 15
- 5.1 Importing Packages . . . 15
- 5.2 Loading Classifier . . . 15
- 5.3 Plate Detection . . . 15
- 5.4 Finding contours . . . 16
- 5.5 Segment Characters . . . 18
- 5.6 CNN Model Configuration . . . 19
- 5.7 Model Building . . . 19
- 5.8 Training and Validation Dataset . . . 21

6 Result and Discussion 22
- 6.1 GUI . . . 22
- 6.2 Experimental Result . . . 25

7 Conclusion 26

References 27

Acknowledgement **29**

List of Figures

1 Working of LPD system 8
2 CNN Model 10
3 Input image for license plate recognition 22
4 Detected number plate region 22
5 Isolated license plate region 23
6 Segmented Characters 23
7 Isolated characters from plate 23
8 Final recognized license plate 24

List of Tables

1 Summary of Recent Research on License Plate Recognition Technologies . . . 5
2 Experimental Results of License Plate Detection and Recognition System . . . 25

1 Introduction

A License Plate Recognition (LPR) System using Convolutional Neural Networks (CNN) is an advanced technology employed for automatically identifying and reading vehicle license plates. This system combines image processing techniques with deep learning, specifically CNNs, to efficiently extract and recognize alphanumeric characters from license plates. The LPR process typically consists of several stages: image acquisition, pre-processing, license plate localization, character segmentation, and character recognition. CNNs, known for their ability to automatically detect features from images, play a crucial role in both license plate detection and character recognition.

In the first stage, the image of the vehicle is captured using a camera. Pre-processing techniques like noise reduction, grayscale conversion, and contrast enhancement are then applied to make the license plate more distinct. The next step involves localizing the license plate in the image using methods like edge detection or contour-based techniques, followed by segmentation, which isolates each character on the plate.

CNNs come into play during the recognition phase, where they are trained on large datasets of labeled license plate characters. The network automatically learns to recognize patterns and features, making it highly accurate in reading the license plate's characters, even under challenging conditions such as low lighting, skewed angles, or blurry images. The CNN model typically comprises multiple convolutional layers, pooling layers, and fully connected layers, enabling it to extract hierarchical features from the image. This deep learning approach has proven to be robust, adaptable, and scalable, making it ideal for real-time applications like toll collection, traffic law enforcement, and parking management systems. Overall, CNN-based LPR systems are transforming the way vehicle identification is automated, offering high accuracy, efficiency, and reliability. .

1.1 Existing Systems

Existing license plate recognition (LPR) systems can be divided into traditional and modern approaches. Traditional systems use optical character recognition (OCR) to identify characters from images of license plates. They usually involve steps like image filtering and edge detection to make the plates clearer before matching the characters. Modern systems, on the other hand, use convolutional neural networks (CNNs) to automatically learn features from images, improving accuracy. The synthetic data and a dynamic random perturbation mechanism further enhances the model's generalization ability. The model can elevate performance while maintaining a low computational complexity. Extensive experiments indicate the approach outperforms state-of-the-art methods on the CCPD dataset and demonstrates robust generalization capabilities on the CLPD and CRPD datasets.[1]

To improve LPR systems, several areas can be focused on. First, using a wider variety of images captured in different lighting, angles, and distances can help make the model more reliable. Optimizing the system for real-time processing is also important, as this can speed up how

quickly it recognizes plates. It's essential to ensure that the system can handle different license plate styles, especially in regions where they vary widely. Improving the OCR part with better algorithms can lead to more accurate character recognition. Additionally, connecting the LPR system to a database for real-time vehicle identification can enhance its usefulness. Improving the user interface will also help users interact better with the system and understand the results. Finally, thorough testing under various conditions and making the model's decisions easier to understand can help identify and fix errors. In future, other deep learning approaches can be implemented which can recognize the LPs if the images contains more than two vehicles.[2]

1.2 Limitations of Existing system

1. Environmental Variability: Many LPR systems struggle with variations in lighting conditions, such as glare from sunlight or low light at night. This can affect the clarity of the license plate and hinder recognition accuracy. The image captured from the scene may be complicated by the camera's type, resolution, lighting aids.[2]

2. Different Plate Formats: Existing systems may have difficulty recognizing license plates that follow different formats, designs, or languages. The image captured from the scene may be complicated by the camera's type, resolution, lighting aids. If the system isn't trained on a diverse set of plates, it may fail to identify them correctly.[3]

3. Image Quality: The performance of LPR systems heavily relies on the quality of the input images. Blurry, distorted, or low-resolution images can lead to misrecognition or failure to detect the plate altogether. Additional types of blur can degrade the quality of licence plates.[2]

4. Occlusions: Plates that are partially obscured by objects, dirt, or damage can be challenging for LPR systems. Existing models may not be robust enough to handle such situations effectively. The types of LP images captured are more diverse and challenging. In the scenarios, which include snow or dust occlusion, low resolution, and the proportion of license plate background occupies too large.[1]

5. Real-Time Processing Challenges: While many modern systems aim for real-time recognition, computational limitations can affect processing speed, especially with high-resolution images or complex models. Real-time processing is essential for LPR systems to operate in dynamic traffic scenarios. Running most of LPR methods on embedded devices remains a challenging problem.[4]

6. Dependence on Training Data: LPR systems require large, well-annotated datasets for training. If the training data is not representative of real-world conditions, the model may not perform well in practical applications. A test dataset is used to assess the system's

accuracy and efficiency to ensure it satisfy the necessary specifications and can handle real-world situations.[5]

7. False Positives and Negatives: Many existing systems experience issues with false positives (incorrectly identifying a plate) and false negatives (failing to recognize a plate), leading to reliability concerns. The recognition accuracy is computed at the plate level. That means a LP is considered as correct if all the characters of this LP are correctly predicted.[6]

8. Security and Privacy Concerns: The use of LPR systems raises concerns about data security and privacy, particularly regarding how license plate data is stored and shared. These technical limitations underscore the necessity for further research endeavors aimed at enhancing the performance and resilience of ALPR systems, all while being mindful of privacy and data security concerns within a scientific context.[7]

9. Cost and Complexity: Implementing advanced LPR systems can be costly and technically complex, requiring significant resources for hardware, software, and maintenance. For license plate localization, which had the capability of detecting license plates that had different variations like rotation, skewness, and different orientations, this method had high computational complexity.[8]

Addressing these limitations can lead to more accurate, reliable, and user-friendly license plate recognition system.

1.3 Proposed system

The proposed license plate recognition system using a Convolutional Neural Network (CNN) involves three primary stages: license plate detection, character segmentation, and character recognition. In the detection phase, the system captures the image of a vehicle and identifies the location of the license plate using image processing techniques such as edge detection or bounding box extraction. Once the license plate is detected, it is cropped and passed to the segmentation stage, where individual characters on the plate are isolated using techniques like contour detection.

1.4 Project Scope

The scope of a License Plate Recognition (LPR) System project utilizing Convolutional Neural Networks (CNNs) encompasses several key components. Initially, the project aims to define its objectives, such as automating vehicle access in parking facilities, monitoring traffic, or facilitating toll collection. Data collection involves gathering a diverse dataset of license plate images, ensuring variations in country, font, color, and environmental conditions. This dataset

is then preprocessed to include image labeling, augmentation for model robustness, and normalization. The core of the project focuses on selecting and training a suitable CNN architecture, potentially incorporating pre-trained models for improved efficiency. Following training, the model's performance is evaluated using metrics like precision, recall, and F1 score on separate validation and testing datasets. Once optimized, the system will be deployed for real-time processing, integrating with camera feeds to capture and analyze vehicle images. A user interface will be developed to display results and manage configurations. Finally, the project will include plans for ongoing maintenance, retraining with new data, and thorough documentation to facilitate future improvements and ensure system reliability. The snapshot of the vehicle is inputted as image from which the license plate number is detected. The system only recognize the English language license plates.

2 Literature Review

In this chapter, we discuss the summary of research paper surveyed, discussion on papers reviewed, research gaps and finally problem statement.

2.1 Summary of research papers

Table 1: Summary of Recent Research on License Plate Recognition Technologies

Author and Year	Methods	Result	Future Scope
Yang Yang Lee et al[9] 2022	CNN, SSD, YOLOv3	Recognition rate: 99%	Develop a more reliable, accurate, and efficient system.
Hanae Moussaoui et al[5] 2024	YOLOv8, OCR, CVAT tool	Detection accuracy: 99%, Character recognition accuracy: 98%	Address more challenging scenarios.
Khurram Khan et al[8] 2021	Faster R-CNN, LUT, MCT	Detection rate: 96.72%, Recognition rate: 98.02%	Focus on developing a parallel version of the algorithm.
Anmol Pattanaik et al[2] 2023	GAN, IBA, CCA models	Precision: 0.995, Recall: 0.961, F1-score: 0.991	Implement deep learning approaches for license plate recognition.
Anjali Gautam et al[3] 2023	CNN, OCR, CCPD dataset, MobileNetV2	License plate recognition accuracy: 99.7%	Explore advanced OCR techniques to improve reading accuracy.
Yaoting He et al[1] 2024	CRPD, CLPD	Recognition accuracy: CCPD: 99.61%, CLPD: 87.18%, CRPD: 84.44%	Optimize network architecture for real-time deployment.
Rezki Zibani et al[7] 2024	CNN, YOLOv5	Localization accuracy: 98.90%, Segmentation accuracy: 98.10%, Recognition accuracy: 97.20%	Create a comprehensive dataset of license plates from various regions.
Qingwang Wang et al[4] 2024	CNN, YOLOv5	Detection accuracy: 99.3%, Speed: 89 FPS	Incorporate thermal imaging for nighttime detection.
Wei Pan et al[10] 2023	VMMR-Net, VColor-Net	Recognition accuracy: VMMR-Net: 97.18%, VColor-Net: 93.78%	Collect more diverse data to improve recognition accuracy.
Asadollah Shahbahrami[11] 2023	YOLOv3	Localization accuracy: 96.16%	Implement additional processes on GPU platforms.

2.2 Research Gap

1. Robustness to environmental conditions: There is need for CNN models that are more robust to the conditions like poor lighting, shadows, fog, rain or over exposure without requiring extensive prepossessing or image enhancement.

2. Low quality and blurred images: Developing CNN architecture that can effectively handles noisy or low quality inputs is a significant gap. Enhancing super resolution techniques combined with LPR is also under research.

3. Real-time performance: Efficient CNN models that are lightweight yet capable of real-time performance with high accuracy are still lacking, particularly for mobile and embedded systems.

4. Multi-Lingual license plates: There is a need for CNN models capable of recognising multi-lingual or region-specific license plates including symbols, without significant changes in architecture.

2.3 Problem Statement

The increasing number of vehicles on roads has created a demand for automated system that can efficiently manage tasks such as traffic monitoring, parking access control and law enforcement. This Project is to develop a license plate recognition system using Convolutional Neural Network (CNN) that can accurately detect and recognize license plates from images.

2.3.1 Project Goals

1. **Project Goal 1:** The system should reliably detect the presence of license plates within images, even in challenging conditions like low light, occlusion, or angle variations.

   Objective: Once detected, the system should be able to accurately extract the license plate region from the image, minimizing errors due to noise, blurring, or other image degradation.

2. **Project Goal 2:** For practical applications, the system should be able to process images, ensuring timely recognition and extraction of license plate information.

   Objective: The CNN model should be capable of accurately recognizing individual characters within the license plate, including letters, numbers, and special symbols, with minimal errors.

3 Methodology

In this chapter we discussed about block diagram, the overall process, the datasets used for license plates recognition.

3.1 System Architecture

The different modules in design of license plate recognition system are as follows:

1. **The Plate localization:** This first step, known as the detection or the localization phase. Its principal goal is to define the boundaries of the license plate and extract it from the initial image, which contains the vehicle with its background. In this phase, appropriate algorithms are used to determine the exact coordinates of the sub-image region that contains only the license plate. To sum up, this phase returns a sub-image (the license plate image) from the initial vehicle image.

2. **Character Segmentation:** Character segmentation is a critical step in LPR systems that involves isolating individual characters from the detected license plate image. This segmentation is essential for accurate character recognition, as CNN models are designed to process individual characters effectively.

   Techniques for character segmentation:

   (a) Thresholding:

   1.Global Thresholding: A single threshold value is applied to the entire image to separate foreground (characters) from background.

   2.Adaptive Thresholding: The threshold value is adjusted based on local image characteristics to handle varying lighting conditions.

   (b) Edge Detection Edge detection to identify potential character boundaries.

   (c) Morphological Operations: Apply erosion and dilation to refine the segmented regions and remove noise.

   (d) Projection Profiles: Use horizontal and vertical projections to verify the accuracy of the segmentation and identify individual characters.

   (e) Character Extraction: Extract the individual characters from the segmented regions for further processing.

3. **Character Recognition:** Character recognition is the final crucial step in a License Plate Recognition (LPR) system. Once individual characters have been extracted from the segmented license plate image, character recognition aims to accurately identify and classify each character, thereby determining the actual license plate number.

The working of license plate recognition system is depicted in following figure:

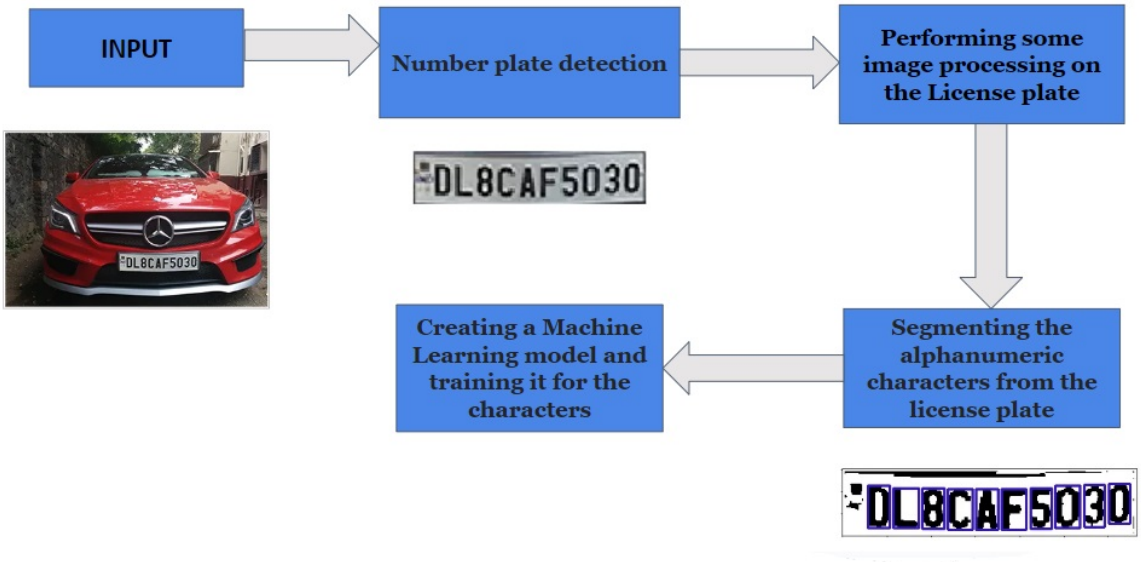


Figure 1: Working of LPD system

3.2 Process Description

1. Number plate detection: License plate detection is the initial step in a License Plate Recognition (LPR) system. It involves identifying the location of the license plate within an image or video frame.

2. Performing some image processing on the License plate:

   The above function takes in the image as input and performs the following operation on it-

   - resizes it to a dimension such that all characters seem distinct and clear
   - convert the colored image to a grey scaled image i.e instead of 3 channels (BGR), the image only has a single 8-bit channel with values ranging from 0–255 where 0 corresponds to black and 255 corresponds to white. We do this to prepare the image for the next process.
   - now the threshold function converts the grey scaled image to a binary image i.e each pixel will now have a value of 0 or 1 where 0 corresponds to black and 1 corresponds to white. It is done by applying a threshold that has a value between 0 and 255, here the value is 200 which means in the grayscaled image for pixels having a value above 200, in the new binary image that pixel will be given a value of 1. And for pixels having value below 200, in the new binary image that pixel will be given a value of 0.
   - The image is now in binary form and ready for the next process Eroding. Eroding is a simple process used for removing unwanted pixels from the object's boundary meaning pixels that should have a value of 0 but are having a value of 1. It works

by considering each pixel in the image one by one and then considering the pixel's neighbor (the number of neighbors depends on the kernel size), the pixel is given a value 1 only if all its neighboring pixels are 1, otherwise it is given a value of 0.

- The image is now clean and free of boundary noise, we will now dilate the image to fill up the absent pixels meaning pixels that should have a value of 1 but are having value 0. The function works similar to eroding but with a little catch, it works by considering each pixel in the image one by one and then considering the pixel's neighbor (the number of neighbors depends on the kernel size), the pixel is given a value 1 if at least one of its neighboring pixels is 1.
- The next step now is to make the boundaries of the image white. This is to remove any out of the frame pixel in case it is present.
- Next, we define a list of dimensions that contains 4 values with which we'll be comparing the character's dimensions for filtering out the required characters.
- Through the above processes, we have reduced our image to a processed binary image and we are ready to pass this image for character extraction.

3. Segmenting the alphanumeric characters from the license plate:

   In this step, we will be applying some more image processing to extract the individual characters from the license plate. The steps involved will be-

   - Finding all the contours in the input image. The function cv2.findContours returns all the contours it finds in the image. Contours can be explained simply as a curve joining all the continuous points (along the boundary), having the same color or intensity.
   - After finding all the contours we consider them one by one and calculate the dimension of their respective bounding rectangle. Now consider bounding rectangle is the smallest rectangle possible that contains the contour. Let me illustrate the bounding rectangle by drawing them for each character here.
   - Since we have the dimensions of these bounding rectangle, all we need to do is do some parameter tuning and filter out the required rectangle containing required characters. For this, we will be performing some dimension comparison by accepting only those rectangle that has a width in a range of 0, (length of the pic)/(number of characters) and length in a range of (width of the pic)/2, 4*(width of the pic)/5. If everything works well we should have all the characters extracted as binary images.
   - The characters may be unsorted but don't worry, the last few lines of the code take care of that. It sorts the character according to the position of their bounding rectangle from the left boundary of the plate.

4. Creating a Machine Learning model and training it for the characters:

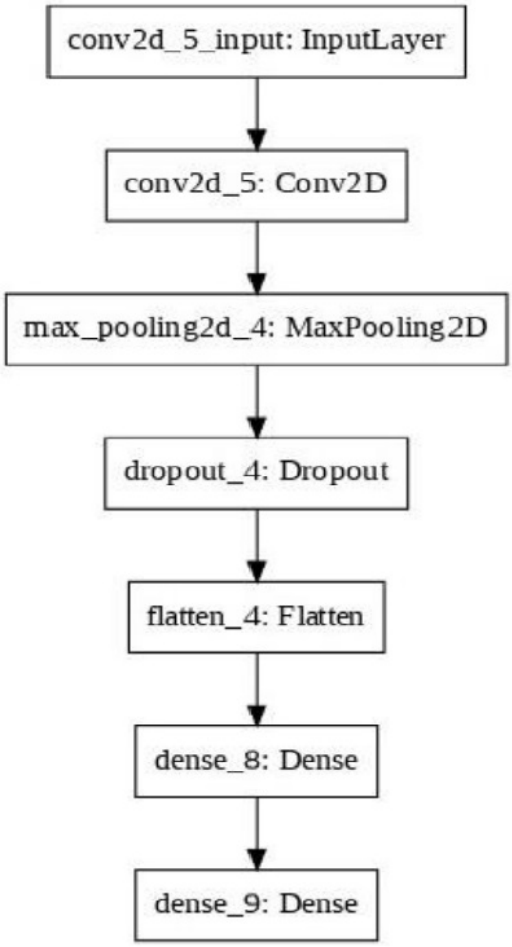


Figure 2: CNN Model

- The data is all clean and ready, create a Neural Network that will be intelligent enough to recognize the characters after training.
- To keep the model simple, start by creating a sequential object.
- The first layer will be a convolutional layer with 32 output filters, a convolution window of size (5,5), and 'Relu' as activation function.
- Next, we'll be adding a max-pooling layer with a window size of (2,2). Max pooling is a sample-based discretization process. The objective is to down-sample an input representation (image, hidden-layer output matrix, etc.), reducing its dimensionality and allowing for assumptions to be made about features contained in the sub-regions binned.
- Now, adding some dropout rate to take care of overfitting. Dropout is a regularization hyperparameter initialized to prevent Neural Networks from overfitting. Dropout is a technique where randomly selected neurons are ignored during training. They are "dropped-out" randomly. We have chosen a dropout rate of 0.4 meaning 60
- Now to flatten the node data so we add a flatten layer for that. The flatten layer takes data from the previous layer and represents it in a single dimension.
- Finally, we will be adding 2 dense layers, one with the dimensionality of the output space as 128, activation function='relu' and other, our final layer with 36 outputs for categorizing the 26 alphabets (A-Z) + 10 digits (0–9) and activation function=' softmax'

5. Training our CNN model:

   - The data will be using contains images of alphabets (A-Z) and digits (0–9) of size 28x28, also the data is balanced so we won't have to do any kind of data tuning here.
   - We'll be using ImageDataGenerator class available in keras to generate some more data using image augmentation techniques like width shift, height shift.
   - Width shift: Accepts a float value denoting by what fraction the image will be shifted left and right. Height shift: Accepts a float value denoting by what fraction the image will be shifted up and down.
   - Use 'categorical_crossentropy' as loss function, 'Adam' as optimization function and 'Accuracy' as our error matrix.

4 Experimental Setup

4.1 Details About Input to Systems

Input is provided by uploading pre-captured images of vehicles through a user interface. The uploaded image is then processed directly by the system for license plate detection and recognition.

4.2 Performance Evaluation Parameters

When evaluating the performance of a ship detection system using deep learning and satellite imagery, several key metrics are commonly used. These metrics help assess how well the model is performing and identify areas for improvement. Here are the main performance evaluation parameters:

1. Accuracy:
Definition: The ratio of correctly predicted instances (both positive and negative) to the total instances.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}} \tag{1}$$

2. Precision
Definition: The ratio of correctly predicted positive instances to the total predicted positives. It measures the accuracy of positive predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{2}$$

3. Recall (Sensitivity)
Definition: The ratio of correctly predicted positive instances to all actual positives. It assesses the model's ability to find all relevant instances.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{3}$$

4. F1 Score
Definition: The harmonic mean of precision and recall. It provides a single score that balances both the precision and recall, especially useful when dealing with imbalanced datasets.

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

5. Mean Average Precision (mAP)
Definition: This metric is often used in object detection tasks. It calculates the average precision across different classes and various intersection over union (IoU) thresholds.
Formula: The specific calculation may vary, but it typically involves:

- Computing the average precision for each class at different IoU thresholds.
- Taking the mean of these average precision values across all classes.

6. Intersection over Union (IoU)
Definition: A measure of overlap between the predicted bounding box and the ground truth bounding box. It is crucial in evaluating how accurately the model predicts the location of ships.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{5}$$

Threshold: A common threshold for a positive prediction is often set at 0.5 or higher.

4.3 Software and Hardware Setup

- Software Setup:
  1. Python Environment: python 3.4.0.14v
  2. Required Libraries:
     - NumPy: For numerical computations and array operations.
     - OpenCV: For image processing tasks like reading, writing, and manipulating images
     - Matplotlib: For visualizing data and results
     - Keras : Providing a simplified interface for building and training neural networks
     - Additional Tools:
       Jupyter Notebook or Google Colab for interactive development and experimentation
- Hardware Requirements:
  1. Computer or Laptop:
     - Processor: Intel Core: i7-7500U
     - RAM: 16GB, regular computer or laptop with a decent CPU and at least 8 GB of RAM.

- GPU: NVIDIA GeForce 920MX, GM108, dedicated GPU is beneficial for faster processing but not essential for small-scale projects.

2. Storage: Sufficient local storage (HDD or SSD) to save uploaded images and any processed data. This can be minimal (a few GBs) depending on the number of images.

3. Network Connection: A stable internet connection if you plan to access online resources, such as pretrained models or cloud storage.

listings

5 Implementation

In this chapter, we elaborate the important parts of source code.

5.1 Importing Packages

```
import matplotlib.pyplot as plt
import numpy as np
import cv2
import tensorflow as tf
from sklearn.metrics import f1_score
from tensorflow.keras import optimizers
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.image import
   ImageDataGenerator
from tensorflow.keras.layers import Dense, Flatten, MaxPooling2D
   , Dropout, Conv2D
```

5.2 Loading Classifier

```
plate_cascade = cv2.CascadeClassifier('/content/drive/MyDrive/
   archive.zip (Unzipped Files)/haarcascade_indian_license_plate
   .xml')
```

5.3 Plate Detection

```
   def detect_plate(img, text=''):
   plate_img = img.copy()
   roi = img.copy()
   plate_rect = plate_cascade.detectMultiScale(plate_img,
      scaleFactor=1.2, minNeighbors=7)

   # Find the largest rectangle
   largest_rect = None
   largest_area = 0
   for (x, y, w, h) in plate_rect:
      area = w * h
```

```
            if area > largest_area:
                largest_area = area
                largest_rect = (x, y, w, h)

        # Draw a rectangle around the largest license plate
        if largest_rect:
            x, y, w, h = largest_rect
            plate = roi[y:y+h, x:x+w, :] # Extract the license plate
                region (ROI)
            cv2.rectangle(plate_img, (x, y), (x+w, y+h), (51, 181,
                155), 3)

            # Add text (optional)
            if text != '':
                center_x = x + w // 2
                center_y = y + h // 2
                plate_img = cv2.putText(plate_img, text, (center_x,
                    center_y),
                                        cv2.FONT_HERSHEY_COMPLEX_SMALL, 0.5,
                                            (51, 181, 155), 1, cv2.LINE_AA)

        return plate_img, plate
```

5.4 Finding contours

```
    # Match contours to license plate or character template
def find_contours(dimensions, img) :

    # Find all contours in the image
    cntrs, _ = cv2.findContours(img.copy(), cv2.RETR_TREE, cv2.
        CHAIN_APPROX_SIMPLE)

    # Retrieve potential dimensions
    lower_width = dimensions[0]
    upper_width = dimensions[1]
    lower_height = dimensions[2]
    upper_height = dimensions[3]

    # Check largest 5 or 15 contours for license plate or
        character respectively
    cntrs = sorted(cntrs, key=cv2.contourArea, reverse=True)[:15]
```

```
ii = cv2.imread('contour.jpg')

x_cntr_list = []
target_contours = []
img_res = []
for cntr in cntrs :
    # detects contour in binary image and returns the
        coordinates of rectangle enclosing it
    intX, intY, intWidth, intHeight = cv2.boundingRect(cntr)

    # checking the dimensions of the contour to filter out the
        characters by contour's size
    if intWidth > lower_width and intWidth < upper_width and
        intHeight > lower_height and intHeight < upper_height :
        x_cntr_list.append(intX) #stores the x coordinate of
            the character's contour, to used later for indexing
            the contours

        char_copy = np.zeros((44,24))
        # extracting each character using the enclosing
            rectangle's coordinates.
        char = img[intY:intY+intHeight, intX:intX+intWidth]
        char = cv2.resize(char, (20, 40))

        cv2.rectangle(ii, (intX,intY), (intWidth+intX, intY+
            intHeight), (50,21,200), 2)
        plt.imshow(ii, cmap='gray')

        # Make result formatted for classification: invert
            colors
        char = cv2.subtract(255, char)

        # Resize the image to 24x44 with black border
        char_copy[2:42, 2:22] = char
        char_copy[0:2, :] = 0
        char_copy[:, 0:2] = 0
        char_copy[42:44, :] = 0
        char_copy[:, 22:24] = 0

        img_res.append(char_copy) # List that stores the
```

```
            character's binary image (unsorted)

    # Return characters on ascending order with respect to the x-
       coordinate (most-left character first)

    plt.show()
    # arbitrary function that stores sorted list of character
       indeces
    indices = sorted(range(len(x_cntr_list)), key=lambda k:
       x_cntr_list[k])
    img_res_copy = []
    for idx in indices:
       img_res_copy.append(img_res[idx])# stores character images
           according to their index
    img_res = np.array(img_res_copy)

    return img_res
```

5.5 Segment Characters

```
    # Find characters in the resulting images
def segment_characters(image) :

    # Preprocess cropped license plate image
    img_lp = cv2.resize(image, (333, 75))
    img_gray_lp = cv2.cvtColor(img_lp, cv2.COLOR_BGR2GRAY)
    _, img_binary_lp = cv2.threshold(img_gray_lp, 200, 255, cv2.
       THRESH_BINARY+cv2.THRESH_OTSU)
    img_binary_lp = cv2.erode(img_binary_lp, (3,3))
    img_binary_lp = cv2.dilate(img_binary_lp, (3,3))

    LP_WIDTH = img_binary_lp.shape[0]
    LP_HEIGHT = img_binary_lp.shape[1]

    # Make borders white
    img_binary_lp[0:3,:] = 255
    img_binary_lp[:,0:3] = 255
    img_binary_lp[72:75,:] = 255
    img_binary_lp[:,330:333] = 255
```

```
    # Estimations of character contours sizes of cropped license
       plates
    dimensions = [LP_WIDTH/6,
                  LP_WIDTH/2,
                  LP_HEIGHT/10,
                  2*LP_HEIGHT/3]
    plt.imshow(img_binary_lp, cmap='gray')
    plt.show()
    cv2.imwrite('contour.jpg',img_binary_lp)

    # Get contours within cropped license plate
    char_list = find_contours(dimensions, img_binary_lp)

    return char_list
```

5.6 CNN Model Configuration

```
K.clear_session()
model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', padding='same',
   input_shape=(28, 28, 3)))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Conv2D(64, (3,3), activation='relu', padding='same'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(36, activation='softmax')) # 36 classes: 0-9 & A
   -Z
model.compile(loss='sparse_categorical_crossentropy', optimizer=
   optimizers.Adam(learning_rate=0.0001), metrics=[
   custom_f1score])
model.summary()
```

5.7 Model Building

```
from tensorflow.keras.preprocessing.image import
   ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
```

```
# Set up the data generators
train_datagen = ImageDataGenerator(rescale=1./255)
validation_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
   '/content/drive/MyDrive/archive.zip (Unzipped Files)/data/
      data/train', # Path to your training data
   target_size=(28, 28), # Changed to (28, 28) to match model
      input shape
   batch_size=32, # Increase batch size if needed
   class_mode='sparse' # Change to 'sparse' for multi-class with
       integer labels
)

validation_generator = validation_datagen.flow_from_directory(
   '/content/drive/MyDrive/archive.zip (Unzipped Files)/data/
      data/val', # Path to your validation data
   target_size=(28, 28), # Changed to (28, 28) to match model
      input shape
   batch_size=32, # Match the batch size used for training
   class_mode='sparse' # Change to 'sparse' for multi-class with
       integer labels
)

# Compile your model before training
# Use sparse_categorical_crossentropy for multi-class
   classification with integer labels
model.compile(optimizer='adam', loss='
   sparse_categorical_crossentropy', metrics=['accuracy'])

# Set up callbacks
callbacks = [EarlyStopping(monitor='val_loss', patience=3,
   restore_best_weights=True)]

# Train the model
model.fit(
   train_generator,
   steps_per_epoch=train_generator.samples // 32, # Adjust based
       on your batch size
   validation_data=validation_generator,
   validation_steps=validation_generator.samples // 32,
```

```
    # Adjust based on your batch size
    epochs=80,
    verbose=1,
    callbacks=callbacks
)
```

5.8 Training and Validation Dataset

```
import tensorflow.keras.backend as K
train_datagen = ImageDataGenerator(rescale=1./255,
    width_shift_range=0.1, height_shift_range=0.1)
path = '/content/drive/MyDrive/archive.zip (Unzipped Files)/data
    /data'
train_generator = train_datagen.flow_from_directory(
      path+'/train', # this is the target directory
      target_size=(28,28), # all images will be resized to 28x28
      batch_size=1,
      class_mode='sparse')
# this is the target directory
validation_generator = train_datagen.flow_from_directory(
      path+'/val',
      # all images will be resized to 28x28 batch_size=1,
      target_size=(28,28),
      class_mode='sparse')
```

6 Result and Discussion

6.1 GUI

In this section we show the graphical user interface and output of our project.

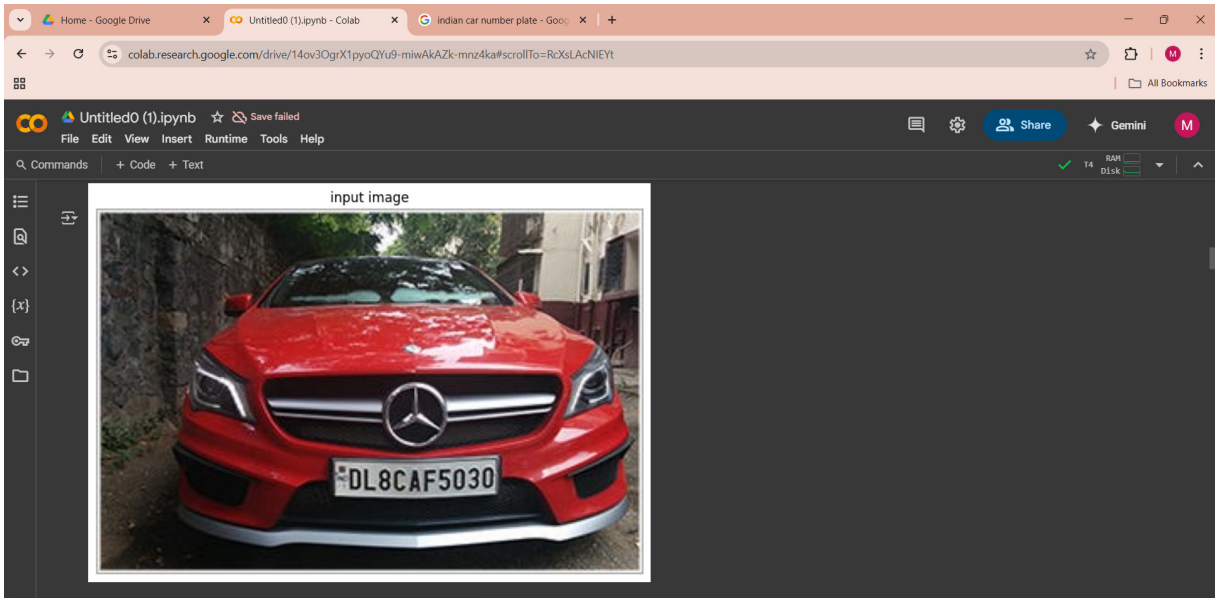


Figure 3: Input image for license plate recognition

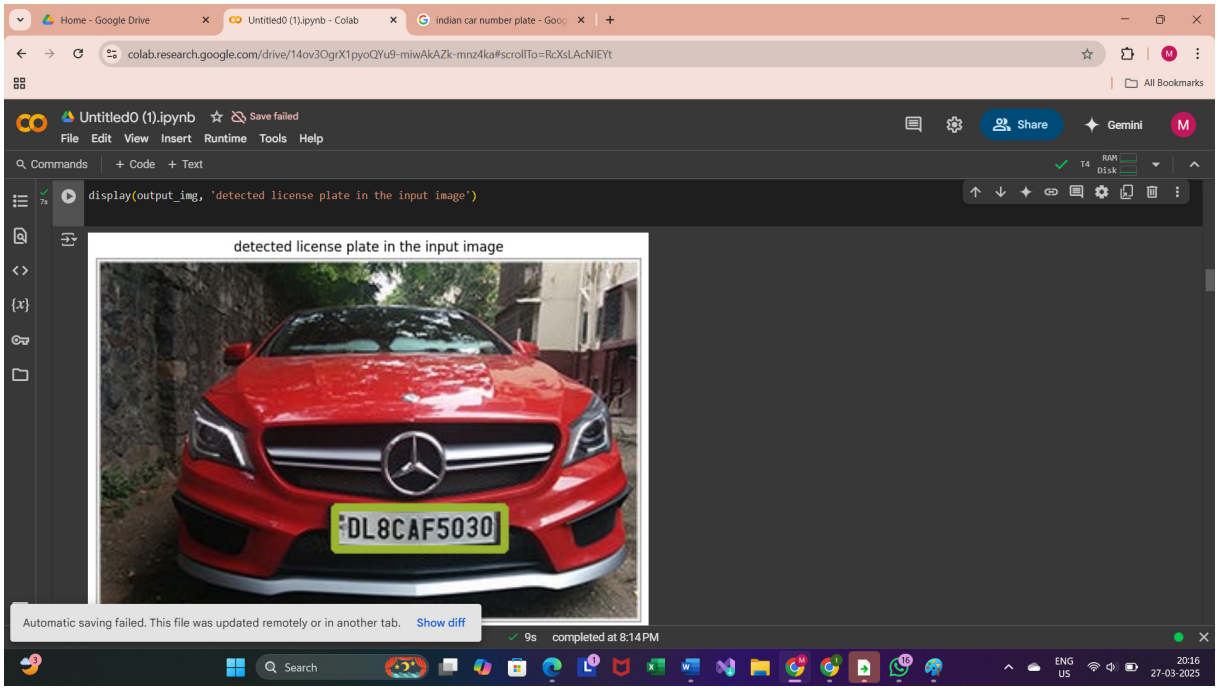


Figure 4: Detected number plate region

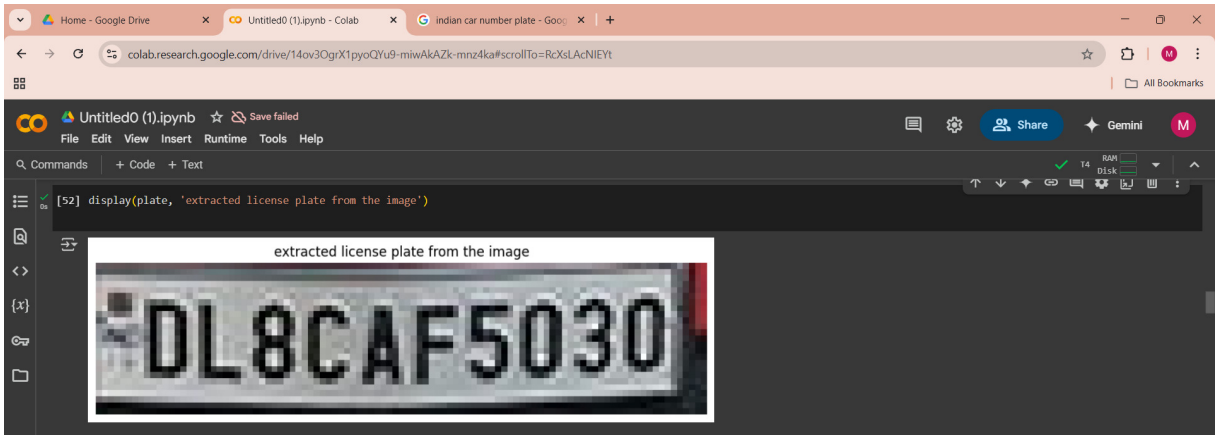


Figure 5: Isolated license plate region

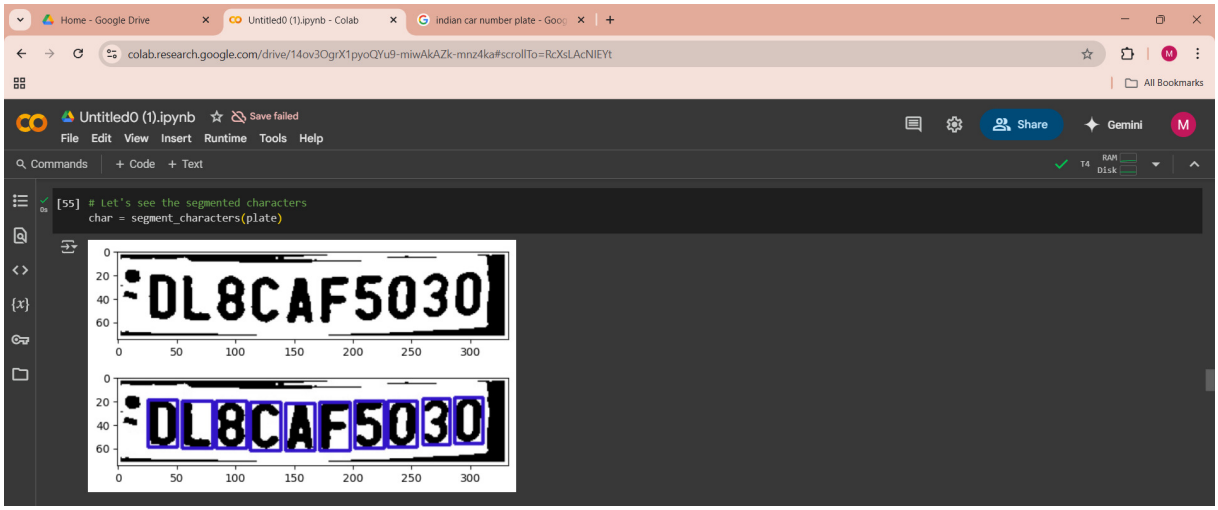


Figure 6: Segmented Characters

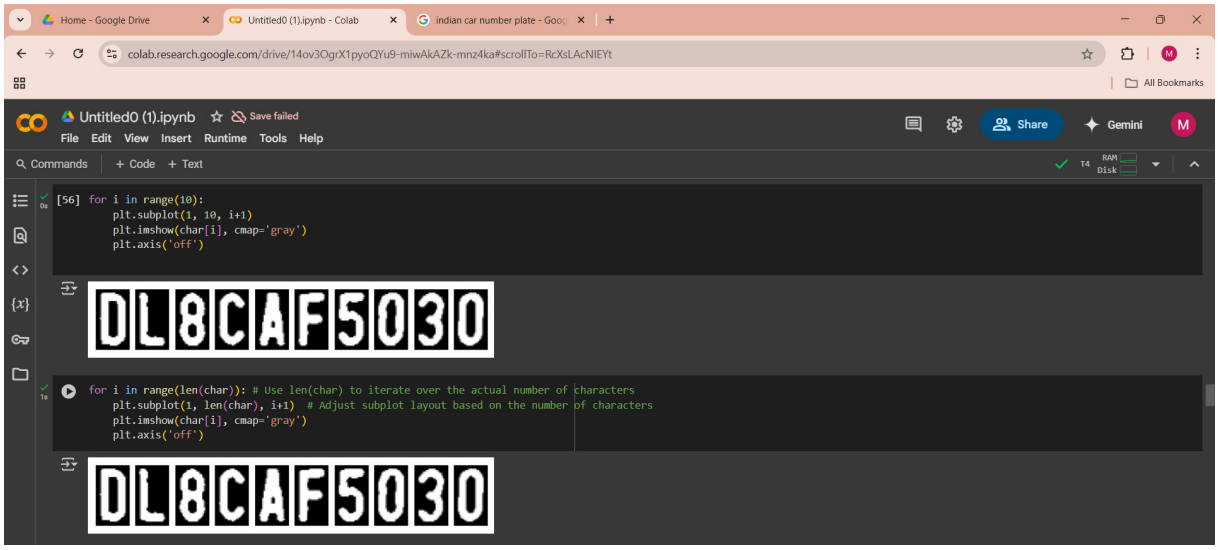


Figure 7: Isolated characters from plate

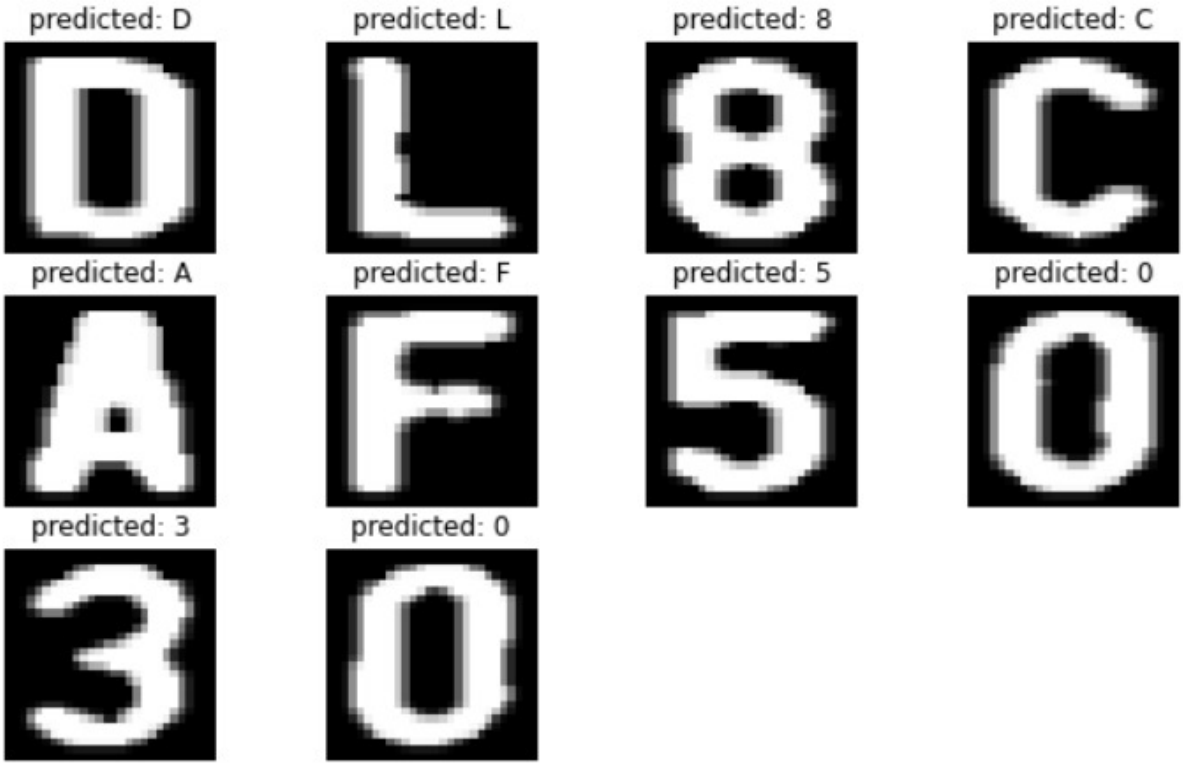


Figure 8: Final recognized license plate

6.2 Experimental Result

Table 2: Experimental Results of License Plate Detection and Recognition System

Sr NO	Car image	Plate Number	Number plate detected	Extracted number plate from image	Correct number plate extracted
1	Car 1	21BH2345AA	YES	21BH2345AA	YES
2	Car 2	DL8CAF5030	YES	DL8CAF5030	YES
3	Car 3	MH20CS1941	YES	MH20CS1941	YES
4	Car 4	HR26DG6167	YES	HR26DG6167	YES
5	Car 5	KL55R2473	YES	KL55R2473	YES
6	Car 6	KL10AV6633	YES	KL10AV6633	YES
7	Car 7	HR11F7575	YES	HR11F7575	YES
8	Car 8	UP16CT2233	YES	UP16CT2233	YES
9	Car 9	HR26DK0830	YES	HR26DK0830	YES
10	Car 10	DL3CAY9324	YES	DL3CAY9324	YES
11	Car 11	HR99BJ6662	YES	HR99BJ6662	YES
12	Car 12	MH03CB6467	YES	WH03CB6467	NO
13	Car 13	MH20BY3665	YES	H20BY3665	NO
14	Car 14	DL12CG6648	YES	DL12GG6648	NO
15	Car 15	KL60N5344	YES	KL60N5344	YES

The experimental results presented in the table evaluate the performance of the proposed License Plate Detection and Recognition system on 15 vehicle images. The system achieved 100% accuracy in detecting the presence of number plates, successfully identifying all 15 images.

In the recognition phase, the system correctly extracted the license plate numbers in 12 out of 15 cases, resulting in an 80% recognition accuracy. In the remaining 3 cases, the system detected the license plates correctly; however, during the extraction phase, it produced incorrect outputs.

7 Conclusion

In this project, we addressed the problem of automatic license plate detection and recognition, a crucial task in traffic monitoring, law enforcement, and smart transportation systems. The goal was to accurately locate and identify license plates from images using a deep learning-based approach, which is essential for applications such as traffic monitoring, vehicle tracking, and automated toll collection.

To achieve this, we used OpenCV's Haar Cascade Classifier for detecting the license plate region from vehicle images. After detection, the extracted plate regions were processed and passed through a Convolutional Neural Network (CNN) model built using TensorFlow and Keras. This model was responsible for recognizing the characters within the detected license plates.

The system was tested on a dataset of 15 input images containing Indian license plates. Out of these, the model correctly detected and recognized the license plates in 12 images, resulting in an effective accuracy of 12 out of 15, or approximately 80. These results demonstrate the system's reliability and effectiveness in license plate detection under typical conditions.

Overall, the project successfully showcased a pipeline for automated license plate detection and recognition using a combination of classical image processing and deep learning techniques. Future improvements could involve enhancing character recognition under low-light or occluded conditions, expanding support for plates from different regions, and integrating real-time video stream processing.

References

[1] Yaoting He, Xin Zhou, Tao Zhou, and Yuanyuan Chen. Alp-net: a segmentation-free approach for license plate recognition in unconstrained scenarios. *Neural Computing and Applications*, 36(23):14559–14574, Aug 2024.

[2] Anmol Pattanaik and Rakesh Chandra Balabantaray. Enhancement of license plate recognition performance using xception with mish activation function. *Multimedia Tools and Applications*, 82(11):16793–16815, May 2023.

[3] Anjali Gautam, Divyesh Rana, Saksham Aggarwal, Swaraj Bhosle, and Hritik Sharma. Deep learning approach to automatically recognise license number plates. *Multimedia Tools and Applications*, 82(20):31487–31504, Aug 2023.

[4] Qingwang Wang, Haochen Song, Zhiyi Liu, Zhimin Tao, and Tao Shen. Cross stage partial dilated convolution network for license plate recognition. *International Journal of Control, Automation and Systems*, 22(6):2029–2037, Jun 2024.

[5] Hanae Moussaoui, Nabil El Akkad, Mohamed Benslimane, Walid El-Shafai, Abdullah Baihan, Chaminda Hewage, and Rajkumar Singh Rathore. Enhancing automated vehicle identification by integrating yolo v8 and ocr techniques for high-precision license plate detection and recognition. *Scientific Reports*, 14(1):14389, Jun 2024.

[6] The-Anh Pham. Effective deep neural networks for license plate detection and recognition. *The Visual Computer*, 39(3):927–941, Mar 2023.

[7] Rezki Zibani, Faouzi Sebbak, Mohamed El Yazid Boudaren, M'hamed Mataoui, Ridouane Hadj Aissa, and Yasser Abdeldjalil Benaissa. Multi-attribute fusion-based approach for algerian automatic license plate recognition. *Multimedia Tools and Applications*, 83(10):30233–30259, Mar 2024.

[8] Khurram Khan, Abid Imran, Hafiz Zia Ur Rehman, Adnan Fazil, Muhammad Zakwan, and Zahid Mahmood. Performance enhancement method for multiple license plate recognition in challenging environments. *EURASIP Journal on Image and Video Processing*, 2021(1):30, Sep 2021.

[9] Yang Yang Lee, Zaini Abdul Halim, and Mohd Nadhir Ab Wahab. License plate detection using convolutional neural network–back to the basic with design of experiments. *IEEE Access*, 10:22577–22585, 2022.

[10] Wei Pan, Xin Zhou, Tao Zhou, and Yuanyuan Chen. Fake license plate recognition in surveillance videos. *Signal, Image and Video Processing*, 17(4):937–945, Jun 2023.

[11] Alireza Akoushideh, Asadollah Shahbahrami, and Abdorreza Joe Afshany. Parallelization of license plate localization on gpu platform. *Multimedia Tools and Applications*, 83:1–14, 05 2023.

Acknowledgement

We sincerely acknowledge with a deep sense of gratitude to Project Guide **Prof. D. P. Mhapasekar** and Project Coordinator **Prof. S. B. Kadam** for their valuable guidance, genuine suggestions, and constant encouragement during the preparation of project report work without which completion of this task would be a difficult task.

We are also thankful to all of our faculty members of **Computer Engineering Department** especially our head of department **Prof. D. P. Mhapasekar** and our respected principal **Dr. D. S. Badkar** who give us an idea of significant cooperation during the completion of this work.

We are immensely grateful to all who were involved in this project work because, without their cooperation, inspiration, constant promotion, and useful suggestions, it would be impossible to complete this task and project report within this allotted time.

Project Members:
Miss. Tejal V. Gawade
Miss. Rutuja S. Parab
Miss.Tejasvi R. Parab

Date: